Michael Walden
March 25, 2021
Udacity Sensor Fusion
Nano-Degree
Mid-Term Report
Project 2

Camera Based 2D Feature Tracking

- **MP.1** The first task was to implement a data buffer that held *dataBufferSize* images within it at a given time and delete the last image in the buffer when a new one needed to be placed within it. This was implemented with a vector of DataFrame type. The new images were pushed onto the buffer with the *push_back* method until the size of the buffer reached *dataBufferSize*. At this point, with each new image that is added to the buffer, all images within the buffer will be shifted by one erasing the last image (the oldest) with a for loop that cycles over all elements and copies the element at *index+1* to the element at index. Then the last image that was pushed onto the buffer is erased with the *pop_back* method and the new image is pushed onto the buffer.

- **MP.2** Task 2 consisted of implementing various detectors including HARRIS, FAST, ORB, AKAZE, and SIFT. These detectors were selectable with a string that can be commented in and out by the developer. Depending on which string the developer un-comments and builds with, a cv::Feature2D pointer object is instantiated and set to the appropriate value based on this string for the modern detectors.

  The Harris Corner Detector creates a *cv::Mat* filled with zeros of the same size as the image. The *cv::cornerHarris* function is called and the result is then normalized. For every pixel in the normalized image, the value is compared to a *minResponse* value and if the pixel is greater than this value then a new *cv::KeyPoint* structure object is filled out with the current point, response, and size. All keypoints are then looped over and the overlap between the *newKeyPoint* and current indexed keypoint is calculated. If this overlap is greater than a *maxOverlap* threshold, the *newKeyPoint* overwrites the currently indexed keypoint. If the calculated overlap isn't greater than the threshold the *newKeyPoint* is pushed onto the end of the *keypoints* vector.

- **MP.3** The third task was to remove all keypoints that fell outside of a predefined *cv::Rect* structure. This rectangular bounding box enclosed the preceding vehicle and removed all keypoints that weren't relevant to the object being tracked. This was accomplished by first creating the *Rect* structure object with the predefined parameters. These parameters consisted of the *width*, *height*, *x* and *y* locations of the top left corner of rectangle. If the flag was set to perform this removal, then a loop iterated over all keypoints testing whether the keypoint fell within the bounding box.

Michael Walden
March 25, 2021
Udacity Sensor Fusion
Nano-Degree
Mid-Term Report
Project 2

If the keypoint did not fall within the bounding box, then the keypoint was erased from the keypoints vector.

- **MP.4** For this task, several descriptors were implemented which included BRIEF, ORB, FREAK, AKAZE, and SIFT. Each individual descriptor was enabled through a string that the developer can comment in and out as needed. Depending on which string was selected in this way, a *DescriptorExtractor* pointer object was instantiated and set to the appropriate value before the compute method was called from this object with the necessary parameters.

- **MP.5** The fifth task consisted of implementing FLANN and k-nearest neighbor. If the developer builds with this string un-commented, a DescriptorMatcher object is instantiated and set to the *cv::DescriptorMatcher::FLANNBASED*. The match method from this object is then called with the descriptors from the source and reference images passed in, and the matches are saved to a vector of type *cv::Dmatch*.

- **MP.6** If SEL_KNN was enabled during compile time, a vector of vectors with type *cv::Dmatch* is instantiated and passed into the matcher method knnMatch with a variable *k* whose value is set to the number of nearest neighbors to find. This vector is then looped through and if the distance of the current indexed vector is less than the distance of the *k*-th member of the current indexed vector multiplied by a *minDescDistRatio* (which is set to 0.8), then the current match is pushed onto the matches vector. Basically looping through all matches and finding its k nearest neighbors and only keeping the matches that meet this distance ratio requirement.

- **MP.7** The seventh task required me to find the number of detected keypoints for each image with each detector type. The results are listed in the table below.

Number of Detected Keypoints											
Detector	Image 1	Image 2	Image 3	Image 4	Image 5	Image 6	Image 7	Image 8	Image 9	Image 10	Total
Shi-Tomasi	1370	1301	1361	1358	1333	1284	1322	1366	1389	1339	13423
Harris	115	98	113	121	160	383	85	210	171	281	1737
FAST	5063	4952	4863	4840	4856	4899	4870	4868	4996	4997	49204
BRISK	2757	2777	2741	2735	2757	2695	2715	2628	2639	2672	27116
ORB	500	500	500	500	500	500	500	500	500	500	5000
AKAZE	1351	1327	1311	1351	1360	1347	1363	1331	1358	1331	13430
SIFT	1438	1371	1380	1335	1305	1369	1396	1382	1463	1422	13861

Michael Walden
March 25, 2021
Udacity Sensor Fusion
Nano-Degree
Mid-Term Report
Project 2

- **MP.8** The eighth task required me to find the number of matched keypoints between each frame using the brute-force method and k-nearest neighbor with distance ratio set to 0.8. The results are listed in the table below. *Note: The first image has all zeros because there is no image previous to it for comparison.*

		Number of Matched Keypoints										
Detector	Descriptor	Image 1	Image 2	Image 3	Image 4	Image 5	Image 6	Image 7	Image 8	Image 9	Image 10	Total
Shi-Tomasi	BRISK	0	127	120	123	120	120	115	114	125	112	1076
Shi-Tomasi	BRIEF	0	127	120	123	120	120	115	114	125	112	1076
Shi-Tomasi	ORB	0	127	120	123	120	120	115	114	125	112	1076
Shi-Tomasi	FREAK	0	127	120	123	120	120	115	114	125	112	1076
Harris	BRISK	0	17	14	18	21	26	43	18	31	26	214
Harris	BRIEF	0	17	14	18	21	26	43	18	31	26	214
Harris	ORB	0	17	14	18	21	26	43	18	31	26	214
Harris	FREAK	0	17	14	18	21	26	43	18	31	26	214
FAST	BRISK	0	420	431	408	430	389	417	422	413	401	3731
FAST	BRIEF	0	420	431	408	430	389	417	422	413	401	3731
FAST	ORB	0	420	431	408	430	389	417	422	413	401	3731
FAST	FREAK	0	420	431	408	430	389	417	422	413	401	3731
BRISK	BRISK	0	254	274	276	275	293	275	289	268	260	2464
BRISK	BRIEF	0	254	274	276	275	293	275	289	268	260	2464
BRISK	ORB	0	254	274	276	275	293	275	289	268	260	2464
BRISK	FREAK	0	234	252	257	262	270	252	269	251	237	2284
ORB	BRISK	0	82	93	95	103	101	115	119	118	116	942
ORB	BRIEF	0	91	102	106	113	109	124	129	127	124	1025
ORB	ORB	0	91	102	106	113	109	124	129	127	124	1025
ORB	FREAK	0	45	53	56	65	55	64	66	70	71	545
AKAZE	BRISK	0	162	157	159	154	162	163	173	175	175	1480
AKAZE	BRIEF	0	162	157	159	154	162	163	173	175	175	1480
AKAZE	ORB	0	162	157	159	154	162	163	173	175	175	1480
AKAZE	FREAK	0	162	157	159	154	162	163	173	175	175	1480
AKAZE	AKAZE	0	162	157	159	154	162	163	173	175	175	1480
SIFT	BRISK	0	136	131	121	135	134	129	136	147	156	1225
SIFT	BRIEF	0	137	131	121	135	134	139	136	147	156	1236
SIFT	FREAK	0	136	130	120	134	133	138	135	146	155	1227

Michael Walden
March 25, 2021
Udacity Sensor Fusion
Nano-Degree
Mid-Term Report
Project 2

- **MP.9** For the last task, the processing time for keypoint detection and descriptor extraction was measured and recorded for each combination of detector and descriptor. The total time and average time per frame were also calculated and recorded. These numbers are listed in the table below.

Processing Time (ms)					
Detector	Descriptor	Keypoint Detection	Descriptor Extraction	Total	Average Time per Frame
Shi-Tomasi	BRISK	108.35	15.66	124.01	12.40
Shi-Tomasi	BRIEF	114.29	8.56	122.85	12.28
Shi-Tomasi	ORB	117.31	27.06	144.37	14.44
Shi-Tomasi	FREAK	89.72	236.09	325.81	32.58
Harris	BRISK	125.93	7.99	133.92	13.39
Harris	BRIEF	122.29	6.33	128.61	12.86
Harris	ORB	132.55	25.74	158.30	15.83
Harris	FREAK	109.55	236.83	346.39	34.64
FAST	BRISK	15.48	36.25	51.73	5.17
FAST	BRIEF	15.03	10.97	26.00	2.60
FAST	ORB	15.09	29.13	44.22	4.42
FAST	FREAK	14.92	247.24	262.16	26.22
BRISK	BRISK	322.74	26.36	349.10	34.91
BRISK	BRIEF	312.74	7.69	320.43	32.04
BRISK	ORB	318.19	82.89	401.08	40.11
BRISK	FREAK	324.98	245.33	570.30	57.03
ORB	BRISK	56.51	14.07	70.59	7.06
ORB	BRIEF	64.16	4.10	68.26	6.83
ORB	ORB	53.04	89.09	142.13	14.21
ORB	FREAK	55.98	238.55	294.53	29.45
AKAZE	BRISK	493.67	16.46	510.12	51.01
AKAZE	BRIEF	491.22	6.27	497.48	49.75
AKAZE	ORB	477.20	61.60	538.80	53.88
AKAZE	FREAK	519.62	257.62	777.24	77.72
AKAZE	AKAZE	477.69	409.09	886.78	88.68
SIFT	BRISK	762.98	14.63	777.61	77.76
SIFT	BRIEF	771.71	4.92	776.63	77.66
SIFT	FREAK	775.86	245.45	1,021.31	102.13

Conclusion

From the measured times in the table above, the recommended combination of detector and descriptor are highlighted. According to the tables above, these are the three fastest combinations and also have the most detected and matched keypoints. Therefor the recommended choice for a detector and descriptor combination is the FAST Detector with the BRIEF Descriptor at 2.6 ms per frame on average and 49,204 total detected keypoints with 3,731 of those matched over all ten images.